

통합 생산기술 대시보드 구축

생산기술 직무 지원 포트폴리오

PT 파워트레인 라인 통합 KPI·디지털트윈 대시보드

모든 수치는 실제 스크립트 실행 결과입니다.

00. 개요

Situation

이전에 개별 시뮬레이션 5개와 대시보드 2개를 따로 만든 적이 있다. 각각은 나름의 결과를 냈지만, 막상 "그래서 이 라인의 OEE가 몇 %인가"라는 질문을 받으면 하나의 숫자로 답할 수 없었다. 시뮬레이션마다 출력 형식이 다르고, 어떤 결과는 실측값이고 어떤 결과는 가정치인지 구분이 안 되어 있었기 때문이다.

Task

① 여러 분석 결과를 표준 KPI 계약($OEE=A \times P \times Q$)으로 통합한다. ② 실측 MTBF/MTTR을 가상라인 시뮬레이션에 주입해 "버퍼를 늘리면?", "PM 주기를 바꾸면?" 같은 질문에 실행 가능한 시뮬레이션으로 답한다.

Action / Result

STEP 0~13을 거치며 데이터 소싱 → KPI 표준화 → SimPy 디지털트윈 → Streamlit 대시보드 → 배포까지 전 과정을 실제로 실행하고 검증했다. 원래 계획은 기존 시뮬레이션 자산(sim1~sim5)을 재활용하는 것이었지만, 실제 작업 환경에 그 자산이 없어 SimPy 가상라인과 스케줄링 로직을 처음부터 새로 설계했다.

01. 데이터 소싱과 검증

공개데이터 탐색

후보	채택	판단 근거
Azure Predictive Maintenance	채택	설비 100대 텔레메트리(876,100행)·고장(761건) 실측 이력
APS Failure at Scania Trucks	채택	클래스 불균형(pos 1.67~2.34%) — 비용민감 임계값 조정에 적합
AI4I 2020	미채택	sim1~5 재활용 포기하며 검토 후 제외
Bosch Production Line (14.3GB)	검토 안 함	데이터 소싱 후보에 애초 미포함

검증 결과 (실측)

Azure PdM: telemetry 876,100행×6열, failures 761행. 고장 부품 비중 comp2 34.0% / comp1 25.2% / comp4 23.5% / comp3 17.2%. APS Scania: training 60,000행(pos 1.67%), test 16,000행(pos 2.34%). 원본은 data/raw/에 그대로 두고 수정하지 않았다.

02. KPI 표준화 — $OEE = A \times P \times Q$

여러 분석 결과가 각자 다른 형식이면 매번 새로 해석해야 한다. 그래서 모든 결과를 하나의 KPI 데이터 계약으로 강제했다:

scenario_id·source·process·kpi_name·kpi_value·unit·ci_low·ci_high·run_id·timestamp. 19개 kpi_name 화이트리스트, 5중 자동 검증(컬럼 누락·dtype·미등록 이름·비율 범위·CI 역전)을 거친다.

가동률(A) = 가동시간 / 계획시간 · 성능(P) = (이론CT × 생산량) / 가동시간 · 품질(Q) = 양품수 / 총생산수

03. 디지털트윈 시뮬레이션

실측 MTBF/MTTR (STEP 3)

부품	MTBF(h)	MTTR(h)	고장건수
comp1	3,045.16	36.02	192
comp2	2,561.57	23.20	259
comp3(열처리)	2,176.39	24.44	131
comp4(조립)	2,328.66	12.65	179
설비 전체 평균	1,410.72	19.98	—

설비 100대 평균 Availability 98.21%. 기준 시나리오(8시간, 버퍼5, PM없음) 몬테카를로 30회: OEE 99.17%(A=100%, P=99.17%, Q=100%), UPH 59.5, 병목공정 열처리(60초).

OEE 손실 구조 — 100%에서 무엇이 얼마나 깎였는가

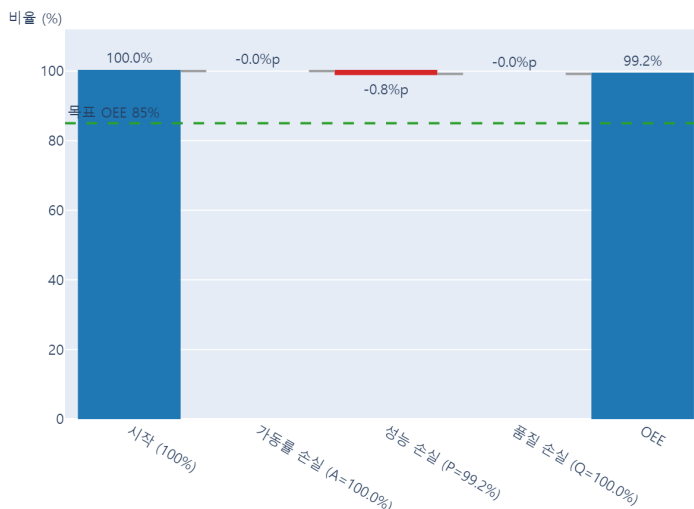


그림 1. OEE 손실 구조 — 시작 100% → 성능손실 0.8%p → 최종 99.2%

시나리오 그리드 — 진짜 발견

버퍼 0~20 전 구간에서 OEE 0.991667로 동일 — 8시간 안엔 실측 MTBF가 커서 고장이 거의 없고, 고장이 없으면 버퍼가 흡수할 변동 자체가 없다. PM 800-1600분에서 Availability 100%로 급등 — 관측기간(480분)보다 PM 주기가 길면 그 창 안에서 PM이 발생하지 않는 당연한 경계 효과(PM 200분 87.5%, 400분 93.1%). 30개 시나리오×30회 몬테카를로, 총 20.3초.

생산계획 — FCFS vs SPT

규칙	메이크스팬
FCFS	575.9초
SPT	1038.6초 (+80.3%)

"SPT가 항상 낫다"는 통념과 반대 결과. SPT를 공정마다 독립 적용하면 처리시간이 긴 특정 잡이 매 공정에서 계속 뒤로 밀려 대기시간이 누적되기 때문 — 현장의 "장기물 방치" 문제와 같은 메커니즘.

생산 스케줄

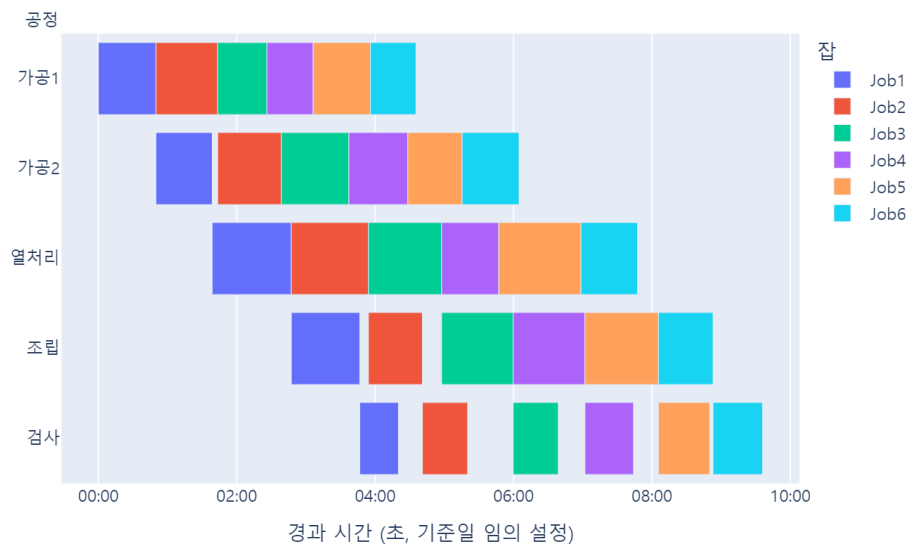


그림 2. FCFS 기준 생산계획 간트차트

비용민감 예지보전 모델 (STEP 8)

모델	임계값	Recall	Precision	총비용
더미(전부 정상)	—	0.0%	—	187,500
기본 RF	0.50	74.4%	83.0%	48,570
비용최적 RF	0.05	99.2%	34.2%	8,670

과검(FP)=10, 미검(FN)=500 비대칭 비용. 더미 대비 178,830(95.4%) 절감. Precision 34.2%는 과검이 여전히 많다는 뜻 — 미검 비용이 훨씬 크다는 전제의 절충.

임계값별 총비용 (과검=10, 미검=500)

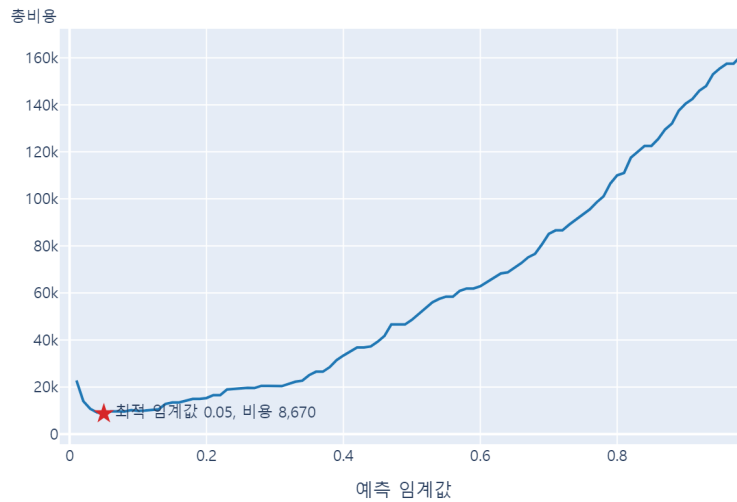


그림 3. 임계값별 총비용 곡선 — 최적 임계값 0.05, 비용 8,670

04. 대시보드 구현 — 캐시 + 실시간 하이브리드

Streamlit 멀티페이지 6개 화면(종합·설비에지보전·품질SPC·생산계획·라인밸런싱·What-if-데이터출처). What-if 시뮬레이터는 자주 쓰는 30개 조합을 미리 계산해 캐시하고, 캐시에 없는 조합만 실시간 계산한다.

방식	소요시간
캐시 조회	2.15ms
실시간 계산	658ms

약 300배 차이 — 대부분의 조작은 즉시 응답하고, 새로운 조합일 때만 명시적으로 "실시간 계산 중"임을 보여준다.

야마즈미 차트 — 공정별 사이클타임 vs Tact

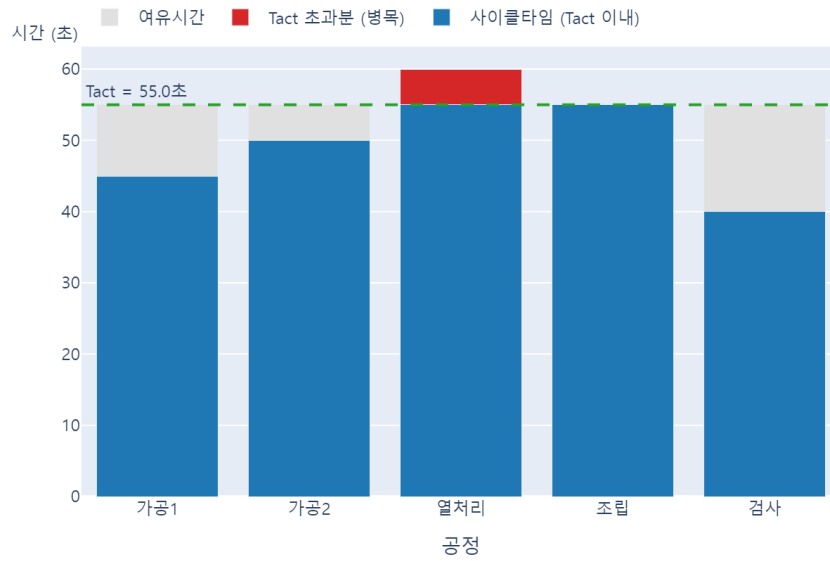


그림 4. 야마즈미 차트 — 공정별 사이클타임 vs 목표 Tact 60초

05. 최종 결과물

배포 URL: <https://weqlag7dcapkjqqjrbgvj.streamlit.app/>

항목	값
기준 시나리오 OEE	99.17% (A=100%, P=99.17%, Q=100%)
비용민감 모델 절감액	178,830 (95.4%), 임계값 0.05
몬테카를로 30회 vs 5회 CI 폭	2.05% vs 7.43%
캐시 vs 실시간	2.15ms vs 658ms (약 300배)
시나리오 그리드 총 소요시간	20.3초 (30개×30회)
FCFS vs SPT 메이크스팬	575.9초 vs 1038.6초 (+80.3%)
페이지 로딩 최적화	10.27초 → 2.19초
배포 데이터 총량	131MB → 약 116KB



그림 5. OEE 게이지 99.2%

파레토 (n=100건)

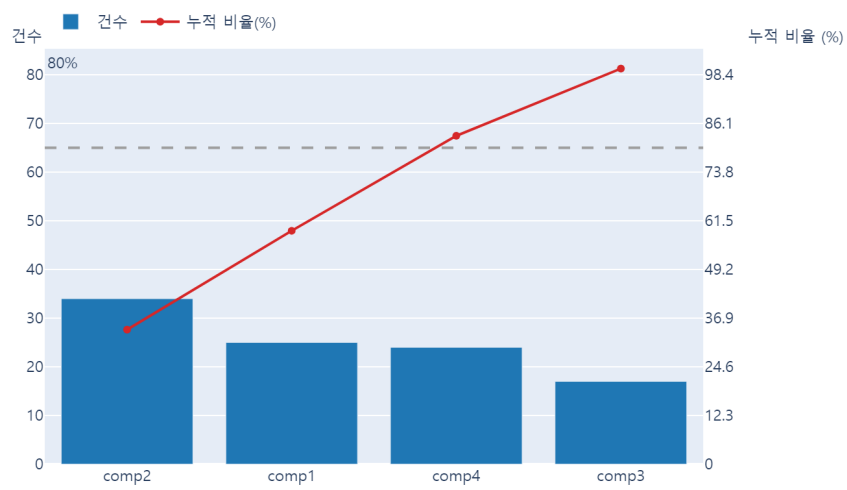


그림 6. 고장 부품 파레토 — comp2 259 / comp1 192 / comp4 179 / comp3 131 (n=761)

X-bar 관리도 (부분군 n=600개)

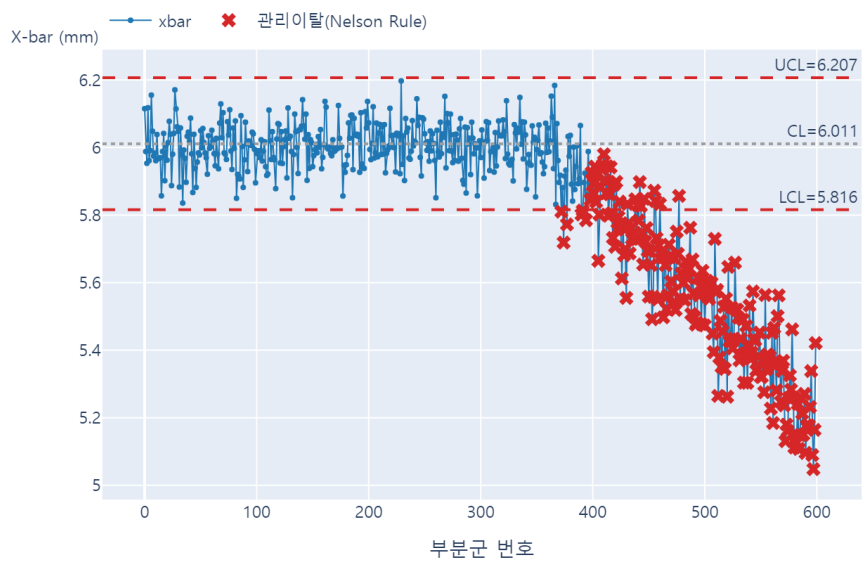


그림 7. 용접 공정 X-bar/R 관리도 (600 부분군, Nelson Rule 위반 208건)

민감도 히트맵

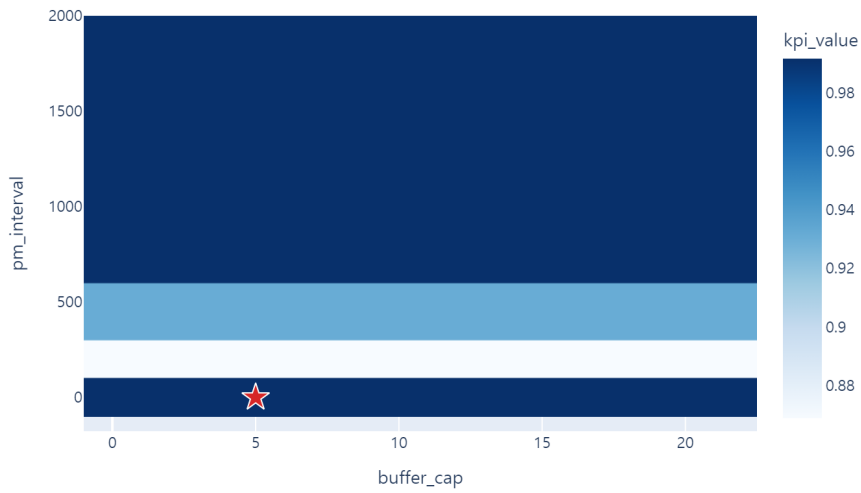


그림 8. 버퍼×PM 민감도 히트맵

몬테카를로 결과 분포 (n=30회)

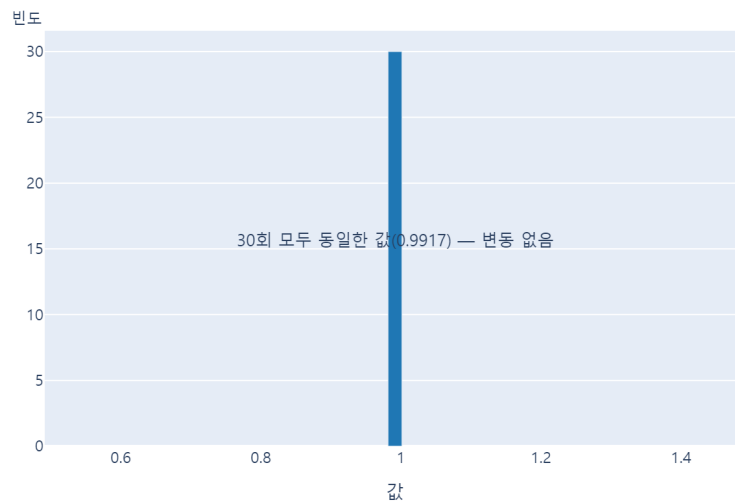


그림 9. 몬테카를로 분포 (변동 없음 케이스, n=30)

06. 인사이트

인사이트 1 — 툴·라이브러리·계산법·깨달은 점

- KPI 계약을 먼저 정의하면 이후 모든 분석이 같은 언어로 말한다.
- 몬테카를로 반복 없이 시뮬레이션 1회만 믿으면 안 된다 — 272일 시뮬레이션에서 기대 고장 3회 중 1회만 발생한 사례를 포아송 근사로 검증(약 15% 확률).

- Kaleido가 받침 있는 한글을 회전 축 제목으로 그릴 때 깨지는 버그를 발견 — 가로 텍스트 주석으로 대체하는 공통 헬퍼로 해결.
- 로컬에서 잘 되던 게 배포하면 깨질 수 있다 — 80MB 원본 CSV 직접 읽기 의존성을 배포 직전 발견·수정.

인사이트 2 — 확장 구상

- MES/PLM 연계 — KPI 계약의 source 컬럼으로 확장 가능하도록 이미 설계.
- 실시간 IIoT 스트리밍 — 배치(24시간 윈도우) 방식을 실시간 알람 구조로 발전 가능.
- 버퍼 효과 재현 — 시뮬레이션 기간을 수백 일로 늘리는 후속 작업 필요(이번엔 미착수).

07. 전체 공정 요약

01 데이터 소싱 → Azure PdM(876,100행, 761건) + APS Scania(pos 1.67~2.34%) 채택

02 KPI 표준화 → $OEE=A \times P \times Q$, 19개 화이트리스트, 5중 검증

03 디지털트윈 → 기준 OEE 99.17% / 버퍼 무효과·SPT 역전 발견 / 비용 95.4% 절감

04 대시보드 → Streamlit 6페이지, 캐시+실시간 300배, 로딩 10.27s→2.19s

05 배포 → 131MB→116KB, Git LFS 불필요, 실배포 버그 3건 발견·수정

부록 A. 생산기술 KPI 매칭표

이 프로젝트에서 다룬 것	생산기술 직무 대응 KPI/업무
OEE 분해, 워터폴 시각화	설비종합효율(OEE) 관리, 손실 구조 분석
MTBF/MTTR 실측 계산	설비보전 지표 관리, 예지보전(PdM)
고장 부품 파레토	불량/고장 유형 분석, 개선 우선순위
비용민감 임계값 조정	예지보전 알람 임계값 설계
SimPy 가상라인, 시나리오	디지털트윈, 라인 시뮬레이션 의사결정
FCFS/SPT 스케줄링	생산계획, 디스패칭 룰 선택
야마즈미 차트	라인밸런싱, Tact Time 관리
X-bar/R, Cp/Cpk	SPC, 품질관리
KPI 데이터 계약	표준화, 데이터 거버넌스

부록 B. 자소서 문장 예시

짧은 버전

여러 시뮬레이션·모델의 결과를 표준 KPI 계약으로 통합하고, 실측 MTBF/MTTR을 SimPy 디지털트윈에 주입해 OEE를 99.17%까지 정량화했습니다. 비용민감 예지보전 모델로 더미 대비 95.4%(178,830) 비용을 절감했고, Streamlit 대시보드를 실제 배포까지 완료했습니다.

긴 버전

이전에 개별적으로 만들었던 시뮬레이션·대시보드가 "OEE가 몇 %인가"라는 질문에 하나의 숫자로 답하지 못한다는 문제의식에서 출발했습니다. 표준 KPI 계약($OEE = \text{가동률} \times \text{성능} \times \text{품질}$)을 먼저 설계하고, Azure Predictive Maintenance의 실측 정비이력(MTBF/MTTR)을 SimPy 기반 5공정 가상라인에 주입해 기준 시나리오 OEE 99.17%를 실측 기반으로 산출했습니다. 버퍼 용량을 0~20까지 바뀌도 8시간 시프트 안에서는 OEE가 전혀 변하지 않는다는 것을 발견했는데, 실측 MTBF가 매우 높아 그 시간 안에 흡수할 변동 자체가 없기 때문이었습니다 — 원인을 규명해 정직하게 문서화했습니다. APS Scania 데이터로는 비용민감 임계값 조정을 통해 더미 모델 대비 95.4%(178,830) 비용을 절감했습니다. Streamlit으로 6개 페이지 대시보드를 만들고 캐시/실시간 하이브리드 구조(300배 응답속도 차이)를 설계했으며, Streamlit Community Cloud에 실제 배포하는 과정에서 로컬에서는 드러나지 않던 원본 데이터 의존성, 패키지 누락 등 3건의 실전 문제를 발견하고 해결했습니다.

부록 C. 트러블슈팅 STAR 정리

STAR 1 — Kaleido의 한글 받침 렌더링 버그

- Situation: PNG 내보내기에서 "공정" 같은 y축 제목이 "꼴0/ㄷ0"처럼 깨져 나왔다.

- Task: 원인을 밝히고 8개 차트 전체에서 재발하지 않게 고쳐야 했다.
- Action: 받침 있는/없는 글자로 최소 재현 코드를 비교해 "받침+90도 회전" 조합에서만 재현됨을 확인, 가로 텍스트 주석으로 대체하는 공통 헬퍼를 만들었다.
- Result: 8개 PNG 전부 정상 렌더링 확인, 이후 모든 차트의 기본값이 됨.

STAR 2 — SPT가 FCFS보다 80% 나쁘게 나온 이유 규명

- Situation: "SPT가 항상 낫다"는 통념과 반대로 SPT 메이크스팬이 80.3% 더 나빴다.
- Task: 로직 버그인지 실제 현상인지 판단해야 했다.
- Action: 개별 잡별 스케줄을 출력해, 특정 잡이 매 공정마다 계속 뒤로 밀리는 패턴을 확인했다.
- Result: 버그가 아닌 정직한 발견으로 대시보드에 그대로 노출했다.

STAR 3 — 배포 직전 발견한 숨은 원본 데이터 의존성

- Situation: 배포 준비 중 설비예지보전 페이지가 80MB 원본 CSV를 직접 읽고 있음을 발견.
- Task: 계산 로직은 바꾸지 않고 원본 의존성만 없애야 했다.
- Action: 계산을 별도 스크립트로 분리해 미리 실행, 결과만 저장하도록 구조 변경 — 수정 전후 계산값 일치 확인.
- Result: 배포 데이터 131MB→116KB로 감소, Git LFS 불필요해짐.